MyCodeSchool

Exerciții teorie — Lecția 01 (rezolvate)

Întrebări de auto-verificare după ce ai terminat exercițiile practice și ai citit `TEORIE.pdf`. Răspunsurile sunt la final — încearcă întâi singur.

Întrebări

1. Ce e `ApplicationContext` și când e construit?

2. Care e diferența funcțională dintre `@Component`, `@Service` și `@Repository`?

3. De ce nu pot pune `@Component` pe `java.time.Clock` ca să-l fac bean?

4. Ce face `@SpringBootApplication`? Înlocuiește câte adnotări și care?

5. De ce e preferată constructor injection în locul `@Autowired` pe câmp?

6. Am o interfață `PaymentGateway` cu două implementări `@Component`. Cer `PaymentGateway` într-un constructor *fără* `@Qualifier`. Ce se întâmplă la pornire?

7. `@Qualifier("email")` vs `@Primary` — care câștigă dacă sunt prezente amândouă?

8. Când rulează metoda adnotată cu `@PostConstruct` — înainte sau după injecția de dependențe? Înainte sau după ca bean-ul să fie folosit?

9. Bean-ul implicit are scope `singleton`. Ce înseamnă concret asta?

10. Ce diferență e între `@Value("${app.port}")` și `@Value("${app.port:8080}")`?

11. Adnotez o clasă cu `@Component` dar Spring nu o găsește la `mvn test`. Care e cel mai probabil motiv?

12. Care e diferența între `NoUniqueBeanDefinitionException` și `NoSuchBeanDefinitionException`?

13. Pot pune `@Bean` într-o clasă `@Component`? Se va comporta la fel ca într-o clasă `@Configuration`?

14. Ce face `@Lazy` peste o clasă `@Component`?

15. Pot avea două `@Bean`-uri cu același nume în context? Ce se întâmplă?

16. Pot face câmpul `private NotificationSender sender` `final` dacă folosesc field injection cu `@Autowired`?

17. Pot pune `@Component` pe o clasă abstractă? Ce se întâmplă la component scan?

18. Ce se întâmplă dacă am dependency circulară — bean-ul A injectează B, iar B injectează A?

19. Cum dau un nume custom unei clase `@Component`? Care e numele implicit dacă nu specific nimic?

20. Ce pot face în constructor vs în `@PostConstruct`? Când aleg unul în detrimentul celuilalt?

21. Am un constructor cu 3 argumente, toate `@Service`-uri. Cum decide Spring ce să injecteze în fiecare?

22. Pot avea 2 metode `@PostConstruct` în aceeași clasă? E o idee bună?

23. Diferența între `@Profile("dev")` și `@ConditionalOnProperty("app.feature.enabled")` — când folosesc fiecare?

24. Cum verific într-un test că un bean *nu* e în context (de exemplu, e activ doar pe un profil pe care testul nu-l setează)?

25. Spring (Framework) vs Spring Boot — care e diferența concretă? Pot folosi Spring fără Boot?

Răspunsuri

1. `ApplicationContext` e registrul tuturor bean-urilor pe care Spring le gestionează. E construit la pornire de `SpringApplication.run(...)` — Spring scanează clasele adnotate, le instanțiază în ordinea corectă a dependențelor și le ține în context până la închiderea aplicației.

2. Funcțional, niciuna. Toate produc același efect: clasa devine bean descoperit la component scan. Diferența e *semantică* — `@Service` indică logică de business, `@Repository` indică acces la date (și în plus traduce excepțiile de persistență la `DataAccessException`). `@Component` e generic. Folosește specializarea care exprimă cel mai bine rolul clasei.

3. Pentru că `Clock` e o clasă din JDK — nu ai acces la codul ei, deci nu poți adăuga o adnotare peste. Soluția e să o publici ca bean dintr-o metodă `@Bean` într-o clasă `@Configuration` pe care o deții.

4. `@SpringBootApplication` e o meta-adnotare care combină trei: `@Configuration` (clasa devine sursă de `@Bean`-uri), `@EnableAutoConfiguration` (Spring Boot configurează automat ce găsește pe

classpath — web, JPA etc.) și `@ComponentScan` (scanează pachetul curent și sub-pachetele).

5. Patru motive principale: (a) dependențele pot fi `final` → imutabilitate; (b) sunt explicite în semnătura constructorului → ușor de văzut ce-i trebuie clasei; (c) poți instanția clasa cu `new` în teste, fără să pornești Spring; (d) imposibil să "uiți" o dependență — nu compilează fără ea.

6. Spring aruncă `NoUniqueBeanDefinitionException` la pornire — vede două bean-uri de același tip și nu știe pe care să-l injecteze. Fix: marchezi pe consumator cu `@Qualifier("nume")`, sau pe una dintre implementări cu `@Primary`.

7. `@Qualifier` câștigă. `@Primary` e doar un default — dacă consumatorul cere explicit un nume prin `@Qualifier`, Spring respectă cererea explicită.

8. După DI, înainte ca bean-ul să fie folosit. Ordinea exactă: (1) Spring construiește bean-ul, (2) injectează dependențele, (3) apelează `@PostConstruct`, (4) bean-ul devine disponibil pentru injecție în alte locuri. Locul ideal pentru: pre-încărcare cache, validări de config, inițializări scumpe.

9. Spring creează o singură instanță a bean-ului pe toată durata vieții containerului. Toți consumatorii care îl cer primesc *același obiect* (referință identică, verificabilă cu `==` sau `assertSame`). Asta înseamnă și că starea internă e partajată — nu pune date mutable în câmpuri instance fără sincronizare.

10. Fără default (`${app.port}`), dacă proprietatea lipsește, Spring eșuează la pornire cu eroare explicită. Cu default (`${app.port:8080}`), dacă lipsește, bean-ul primește `8080`. Default-ul te scapă de crash-uri în dev, dar maschează configurări lipsă în prod — folosește cu grijă.

11. Cel mai probabil clasa e într-un pachet care NU e sub pachetul clasei `@SpringBootApplication`. `@ComponentScan` (implicit prin `@SpringBootApplication`) pornește din pachetul clasei principale. Mută clasa sub el, sau adaugă explicit `@ComponentScan(basePackages = "...")`.

12. `NoSuchBeanDefinitionException` = niciun bean de tipul/numele cerut nu există în context. `NoUniqueBeanDefinitionException` = există *mai multe* bean-uri și Spring nu poate alege. Cauza tipică: ai uitat o adnotare (`NoSuchBean`) vs ai 2+ implementări fără `@Qualifier` / `@Primary` (`NoUnique`).

13. Da — `@Bean` funcționează și într-o clasă `@Component`. Diferența: într-o clasă `@Configuration`, Spring "interceptează" apelurile între metodele `@Bean` ca să garanteze singleton (apelarea `clock()` de două ori în aceeași config returnează același obiect). Într-o clasă `@Component` simplă, asta NU se întâmplă — fiecare apel direct e un apel Java normal. Folosește `@Configuration` pentru claritate.

14. Spring nu creează bean-ul la pornire — îl construiește prima dată când e cerut prin DI sau `getBean`. Util pentru bean-uri scumpe care nu sunt necesare în majoritatea run-urilor. Trade-off: prima cerere e mai lentă; dacă bean-ul are config invalid, eroarea apare la rulare, nu la pornire.

15. Implicit, nu — Spring aruncă eroare la pornire (override-ul de bean e dezactivat by default în Spring Boot). Cauze tipice: două clase cu nume identic în pachete diferite, sau două metode `@Bean` cu același nume. Fix: redenumește una sau dă-i explicit alt nume (`@Bean("alias")` , `@Component("alias")`).

16. Nu. Câmpurile `final` trebuie inițializate în constructor sau în declarație. Field injection se face prin reflection *după* ce obiectul e construit — Java nu permite scriere pe un câmp `final` în acel moment. Asta e încă un motiv (al patrulea, după testabilitate, imutabilitate și dependențe explicite) pentru a prefera constructor injection, care permite `final` .

17. Tehnic, da — adnotarea se aplică. Dar Spring nu poate instanția o clasă abstractă, deci NU publică un bean. Component scan o "vede" și o ignoră. Folosește `@Component` pe clase concrete; dacă ai o ierarhie, marchează doar subclasele care vrei să devină bean-uri.

18. Cu constructor injection (preferat), Spring eșuează la pornire cu `BeanCurrentlyInCreationException` — fiecare bean așteaptă ca celălalt să fie complet construit, dar niciunul nu poate. Cu field/setter injection, *uneori* funcționează (Spring poate amâna), dar e fragil și ascunde un design problematic. Soluția corectă: extragi o a treia clasă/interfață care rupe ciclul, sau combini cele două responsabilități într-o singură clasă.

19. Treci numele ca valoare în adnotare: `@Component("nume-explicit")` . Default-ul (fără nume) e numele clasei cu prima literă minusculă: `EmailSender` → `emailSender` , `UserService` → `userService` . Numele explicit e util când vrei să-l referezi din `@Qualifier("nume-explicit")` (cum facem în Ex04 cu `@Component("email")`).

20. În **constructor** primești dependențele și le poți salva în câmpuri, dar nu le folosi *intens* — alte bean-uri pot fi încă în curs de construcție în lanț. În **@PostConstruct** toate dependențele sunt 100% gata: e momentul pentru validări de config la pornire, pre-încărcare cache, conexiuni externe, sau orice depinde de starea unui alt bean. Regula scurtă: dacă acțiunea necesită "totul e gata", e `@PostConstruct` .

21. Auto, prin tip (type-based autowiring). Pentru fiecare parametru, Spring caută în context un bean al cărui tip e asignabil. Dacă găsește exact unul → injectează. Dacă 0 → `NoSuchBeanDefinitionException` . Dacă 2+ → `NoUniqueBeanDefinitionException` (fix cu `@Qualifier` sau `@Primary`). Pentru un singur constructor public, `@Autowired` e implicit (Spring 4.3+).

22. Da, e legal — Spring le va apela pe toate. Dar **ordinea nu e garantată** (JSR-250 nu o specifică, depinde de implementare). Recomandat: o singură metodă `@PostConstruct` care apelează metodele private în ordinea pe care o vrei. Mai ușor de citit, deterministic, ușor de extins.

23. `@Profile("dev")` activează bean-ul când *profilul activ* include "dev" — folosit pentru variante de **mediu** (dev/test/prod): config DB diferită, mock-uri vs servicii reale. `@ConditionalOnProperty("app.feature.enabled")` activează bean-ul când o **proprietate** are valoarea cerută (de obicei `true` / `false`) — folosit pentru **feature flags** care nu depind de mediu. `@Profile` e mai vechi și mai concis; `@Conditional*` e mai general.

24. Cea mai clară opțiune: `ApplicationContext.getBeansOfType(MyBean.class)` și verifici `.isEmpty()`. Sau injectezi `ObjectProvider<MyBean>` și apelezi `.getIfAvailable()` — întoarce `null` dacă nu există. **Nu** folosi `@Autowired MyBean bean` — wire-up-ul va eșua *înainte* de testul tău, cu o eroare confuză (`UnsatisfiedDependencyException`) în loc de o asertțiune curată.

25. Spring (Framework) = container IoC + DI + module pentru web, persistență, securitate, AOP — toate, configurate manual prin XML sau Java config; tu alegi versiuni, integrezi server, decizi tot. **Spring Boot** = un strat opinionat peste Spring care: (a) include un server embedded (Tomcat) cu config implicită, (b) auto-configurează module bazat pe ce găsește pe classpath, (c) oferă starter dependencies (`spring-boot-starter-web` aduce tot ce-ți trebuie). Da, poți folosi Spring fără Boot — multe proiecte vechi o fac — dar setup-ul e mult mai lung și fiecare module trebuie wiring manual.